

Tarea 7 – Servicios Cognitivos de Azure

Proyecto 2: OCR de Facturas con
Azure Document Intelligence

Antonio Ferrer Martínez

Marzo 2026

Índice

1. Introducción.....	2
2. Arquitectura general.....	2
3. Servicios de Azure utilizados.....	4
3.1. Azure Document Intelligence (Form Recognizer)	4
3.2. Configuración del recurso.....	4
3.3. Paquete NuGet y código de conexión.....	5
3.4. Estructura del AnalyzeResult.....	5
3.5. Diferencias con el ejemplo de clase.....	7
4. Estructura del proyecto.....	8
4.1. Modelo de datos (ResultadoFactura).....	8
4.2. Servicio de base de datos (ServicioBaseDatos).....	9
5. Requisitos e instalación.....	10
5.1. Requisitos previos	10
5.2. Cómo ejecutar el proyecto.....	10
5.3. Configuración de Azure.....	11
6. Funcionalidades principales.....	12
6.1. Procesamiento de facturas	12
6.2. Procesamiento por lotes.....	13
6.3. Historial y detalle	13
6.4. Sistema multidioma.....	13
6.5. Tema claro / oscuro.....	13
7. Capturas de pantalla	14
8. Conclusiones.....	19

1. Introducción

He elegido el Proyecto 2 de la práctica (OCR de Facturas con Azure Document Intelligence). La idea es desarrollar una app .NET MAUI que permita hacer una foto a una factura con la cámara del móvil o elegir un archivo (imágenes, PDFs, XMLs de factura electrónica) y que la app mande ese archivo al servicio de Azure Document Intelligence para extraer los datos importantes: nombre del proveedor, número de factura, fecha e importe total.

Para desarrollar la app me he basado en los materiales de clase, sobre todo en el documento "Creación del recurso Form Recognizer" donde viene explicado paso a paso cómo crear el recurso en Azure y el código de ejemplo en C# con el paquete Azure.AI.FormRecognizer. También he consultado la presentación "Azure y Servicios de IA en la Nube" para entender las distintas categorías de servicios cognitivos y el glosario de términos de Azure para tener claros los conceptos básicos (recursos, grupos de recursos, suscripciones, etc.).

Los datos extraídos se guardan automáticamente en una base de datos SQLite dentro de la app, y se pueden consultar después en la pestaña de historial. También he añadido un sistema de copia de seguridad opcional por si el usuario quiere tener los archivos guardados en otra carpeta.

La app está disponible en 9 idiomas (castellano, inglés, y las lenguas cooficiales de España: catalán, euskera, gallego, aragonés, aranés, asturiano y extremeño) y se puede cambiar el idioma en tiempo real desde los ajustes sin tener que reiniciar la app.

2. Arquitectura general

La arquitectura de la aplicación es relativamente sencilla. El usuario interactúa con la interfaz MAUI (que tiene tres pestañas: Analizar, Archivo y Ajustes), la app manda las imágenes o PDFs al servicio de Azure por HTTPS, y cuando Azure responde con los datos extraídos, la app los formatea, los muestra en pantalla y los guarda en una base de datos SQLite local. Opcionalmente también puede copiar los archivos a una carpeta de backup que elija el usuario.

Arquitectura - FacturasOCR

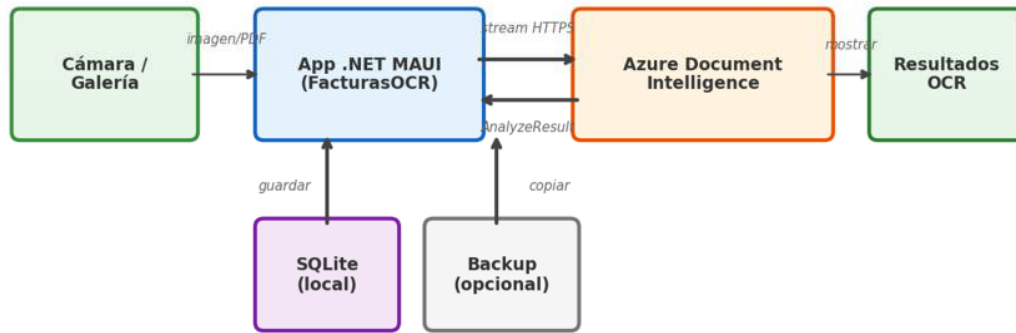


Fig. 1: Esquema de arquitectura de la app

El flujo de procesamiento es el siguiente:

1. El usuario elige una imagen (cámara o galería) o selecciona un archivo PDF/XML.
2. Si es una imagen o PDF, la app manda el archivo al servicio de Azure Document Intelligence por HTTPS usando el modelo prebuilt-invoice.
3. Azure analiza el documento y devuelve un objeto AnalyzeResult con pares clave-valor, campos específicos de factura (VendorName, InvoiceTotal...), páginas con líneas de texto y tablas.
4. La app formatea el resultado, extrae los campos principales y los muestra en una ficha visual.
5. Se guarda todo en la base de datos SQLite local y opcionalmente se copia a la carpeta de backup.
6. Si el archivo es XML (factura electrónica FacturaE), se parsea localmente sin necesidad de Azure.

3. Servicios de Azure utilizados

3.1. Azure Document Intelligence (Form Recognizer)

Azure Document Intelligence (antes se llamaba Form Recognizer, como aparece en los apuntes de la Unidad 8) es un servicio cognitivo de Azure que usa modelos de IA para extraer texto, pares clave-valor, tablas y campos estructurados de documentos. Como se explica en el glosario de Azure de clase, es un servicio dentro de la categoría "Vision" de los Azure Cognitive Services. Es como un OCR pero más avanzado porque no solo lee el texto sino que entiende la estructura del documento y sabe identificar qué información es cada cosa.

En mi app uso el modelo prebuilt-invoice, que es un modelo pre-entrenado específico para facturas. Este modelo sabe buscar automáticamente campos como:

Campo Azure	Descripción
VendorName	Nombre del proveedor/emisor de la factura
InvoiceId	Número de factura
InvoiceDate	Fecha de la factura
InvoiceTotal	Importe total

Además de estos campos, el resultado de Azure también incluye los pares clave-valor genéricos (como se muestra en el ejemplo ShowKeyValuePairsSummary del código de clase), las líneas de texto de cada página (ShowLayoutAndMarksSummary) y las tablas con sus celdas (ShowTablesSummary). Esto es útil porque no todas las facturas tienen el mismo formato y a veces Azure no detecta un campo pero sí lo tiene en los pares clave-valor.

3.2. Configuración del recurso

Para crear el recurso en Azure seguí los pasos del documento "Creación del recurso Form Recognizer" que nos proporcionó el profesor: Datos Básicos, Red (dejé el acceso público porque es un entorno de pruebas), Identity (desactivado), Tags y Revisar y Crear. Usé el tier gratuito F0 que permite hasta 500 páginas al mes. Una vez creado, en la sección "Claves y punto de conexión" copié el Endpoint y la API Key.

Parámetro	Valor
Endpoint	https://uxentio.cognitiveservices.azure.com/
Modelo	prebuilt-invoice
Tier	F0 (gratuito)

3.3. Paquete NuGet y código de conexión

La conexión con Azure se hace con el paquete NuGet Azure.AI.FormRecognizer versión 4.1.0, el mismo que se usa en el ejemplo de clase (FormRecognizer/Program.cs). El patrón es el mismo: crear un DocumentAnalysisClient con el endpoint y la clave, llamar a AnalyzeDocumentAsync y procesar el AnalyzeResult. La diferencia es que en el ejemplo de clase se usa "prebuilt-document" (genérico) y yo uso "prebuilt-invoice" que está optimizado para facturas. El código básico:

```
var client = new DocumentAnalysisClient(
    new Uri(endpoint),
    new AzureKeyCredential(apiKey));

var operation = await client.AnalyzeDocumentAsync(
    WaitUntil.Completed,
    "prebuilt-invoice",
    imageStream);

AnalyzeResult result = operation.Value;
```

El endpoint y la API key se pueden cambiar desde la pantalla de Ajustes de la app, por si el usuario quiere usar sus propias credenciales de Azure.

3.4. Estructura del AnalyzeResult

Cuando Azure termina de analizar un documento, devuelve un objeto AnalyzeResult que contiene toda la información extraída. Es exactamente lo mismo que se ve en el ejemplo de clase (FormRecognizer/Program.cs), donde cada parte se muestra con una función Show diferente. En mi app proceso las mismas partes pero adaptadas a MAUI:

Propiedad	Tipo de datos	Descripción y equivalencia en ejemplo de clase
KeyValuePairs	Pares clave-valor genéricos	Detecta etiquetas y sus valores por proximidad en el documento. Ej: "Fecha" → "15/01/2024". Equivale a ShowKeyValuePairsSummary del ejemplo.
Documents[].Fields	Campos del modelo	Campos semánticos específicos del modelo prebuilt-invoice: VendorName, InvoiceId, InvoiceDate, InvoiceTotal, etc. En el ejemplo de clase (que usa prebuilt-document) esta parte se muestra con ShowEntitiesSummary.
Pages[].Lines	Líneas de texto por página	Todas las líneas de texto detectadas, con sus coordenadas (BoundingPolygon). En el ejemplo se muestra con ShowLayoutAndMarksSummary.
Pages[].Words	Palabras individuales	Cada palabra con su texto y nivel de confianza. Útil para entender qué ha leído Azure exactamente.
Tables[].Cells	Tablas con celdas	Tablas detectadas en el documento, con fila, columna y contenido de cada celda. En el ejemplo se usa ShowTablesSummary. Yo las uso como fallback para buscar totales.

En el ejemplo de clase también se procesan las secciones manuscritas (ShowHandwrittenSectionsSummary) y las marcas de selección (SelectionMarks), pero en mi app no las uso porque las facturas normales no suelen tener texto manuscrito ni checkboxes.

3.5. Diferencias con el ejemplo de clase

El código de ejemplo del profesor (FormRecognizer/Program.cs) y mi app comparten la misma base, pero hay varias diferencias importantes que tuve que ir resolviendo:

- **Modelo utilizado:** El ejemplo usa "prebuilt-document" (genérico, para cualquier documento). Yo uso "prebuilt-invoice" que está optimizado para facturas y devuelve campos como VendorName, InvoiceTotal, etc. directamente en Documents[.Fields.
- **Origen del documento:** El ejemplo analiza un PDF desde una URL (AnalyzeDocumentFromUriAsync). Mi app usa un stream local (AnalyzeDocumentAsync) porque la imagen viene de la cámara o del sistema de archivos del móvil.
- **Formato de salida:** El ejemplo imprime todo por consola. Mi app formatea el resultado en un editor de texto y extrae los campos clave para mostrarlos en una ficha visual con colores.
- **Plataforma:** El ejemplo es una app de consola .NET. Mi app es .NET MAUI con interfaz gráfica, navegación por pestañas, base de datos SQLite y soporte multiidioma.
- **Acceso a campos:** En el ejemplo se usa field.Value?.ToString() que funciona bien con prebuilt-document. Con prebuilt-invoice tuve que usar field.Content porque los campos tienen tipos ricos (Currency, Date) y ToString() no devuelve el valor legible.

4. Estructura del proyecto

El proyecto sigue la estructura típica de una app .NET MAUI, con los archivos organizados en carpetas según su función. He puesto los nombres en español para que sea más fácil de entender:

Archivo	Descripción
MauiProgram.cs	Punto de entrada, configura fuentes y servicios
App.xaml / App.xaml.cs	Aplicación principal, aplica el tema guardado al arrancar
AppShell.xaml / .cs	Navegación por pestañas (Analizar, Archivo, Ajustes)
PaginaPrincipal.xaml / .cs	Pantalla principal: botones, preview de imagen, ficha de datos, procesamiento OCR
Paginas/PaginaHistorial.xaml / .cs	Historial de facturas guardadas con lista
Paginas/PaginaDetalle.xaml / .cs	Detalle de una factura (imagen, campos, texto completo)
Paginas/PaginaAjustes.xaml / .cs	Ajustes: Azure, apariencia (tema), idioma, almacenamiento, backup
Modelos/ResultadoFactura.cs	Modelo de datos / tabla SQLite con campos de factura
Servicios/ServicioAnalisis.cs	Conexión con Azure Document Intelligence
Servicios/ServicioBaseDatos.cs	Servicio de base de datos SQLite (CRUD)
Servicios/ServicioIdiomas.cs	Sistema de traducciones para 9 idiomas

El archivo más grande e importante es `PaginaPrincipal.xaml.cs`, que contiene toda la lógica de procesamiento de facturas: llamada a Azure, formateo del resultado, extracción de campos, parseo de XML, guardado automático y detección de duplicados.

4.1. Modelo de datos (ResultadoFactura)

Cada factura procesada se guarda en una base de datos SQLite local usando la clase `ResultadoFactura` (en `Modelos/ResultadoFactura.cs`). Esta clase tiene un atributo `[Table("InvoiceResult")]` para que la tabla de la BD se llame siempre igual aunque la clase se haya renombrado a español. Los campos son:

Campo	Tipo	Descripción
Id	int	Clave primaria autoincremental. SQLite se encarga de asignarla.
AnalyzedAt	DateTime	Fecha y hora en que se procesó la factura.
ImagePath	string	Ruta donde se guardó la copia local del archivo original (imagen, PDF o XML).
RawText	string	Texto completo del OCR: todos los pares clave-valor, campos, líneas y tablas que devolvió Azure formateados como texto plano.
FileName	string	Nombre del archivo original (ej: "factura_mayo.pdf").
VendorName	string	Nombre del proveedor extraído por Azure o por el fallback manual.
InvoiceId	string	Número de factura extraído.
InvoiceDate	string	Fecha de la factura extraída.
Total	string	Importe total extraído.

Los campos VendorName, InvoiceId, InvoiceDate y Total se guardan como string y no como sus tipos nativos (DateTime, decimal...) porque Azure no siempre los devuelve en un formato estándar. Por ejemplo, una fecha puede venir como "15/01/2024", "January 15, 2024" o "2024-01-15" según el idioma de la factura. Guardarlos como texto evita problemas de parseo.

4.2. Servicio de base de datos (ServicioBaseDatos)

El servicio ServicioBaseDatos.cs maneja toda la comunicación con SQLite. Usa el paquete NuGet sqlite-net-pcl que funciona con async/await. Las operaciones que ofrece son:

- SaveInvoiceAsync: guarda o actualiza una factura. Si el Id es 0 hace INSERT, si no UPDATE.

- GetAllInvoicesAsync: devuelve todas las facturas ordenadas por fecha (las más recientes primero).
- GetInvoiceAsync: busca una factura por su Id para mostrar en la pantalla de detalle.
- DeleteInvoiceAsync: elimina una factura de la BD.
- FindDuplicateAsync: busca si ya existe una factura con el mismo nombre de archivo o con contenido similar (comparando los primeros 200 caracteres del texto). Esto se usa para evitar guardar duplicados.

La ruta de la base de datos se puede cambiar desde los ajustes de la app. Por defecto se guarda en una carpeta "DatosFacturas" junto al ejecutable, pero el usuario puede elegir cualquier carpeta (por ejemplo el Escritorio). Si se cambia la ruta, la app detecta que ha cambiado y reconecta a la nueva BD automáticamente.

5. Requisitos e instalación

5.1. Requisitos previos

Para poder compilar y ejecutar el proyecto se necesita:

- **Visual Studio 2022 (17.8 o superior):** Con la carga de trabajo ".NET Multi-platform App UI development" instalada.
- **.NET 9 SDK:** Se puede descargar desde <https://dotnet.microsoft.com/download>. El proyecto usa net9.0.
- **Windows 10 (build 17763 o superior):** Para ejecutar la versión Windows de la app. También se puede ejecutar en Android con un emulador.
- **Cuenta de Azure con recurso Document Intelligence:** Necesario para que el OCR funcione. Se puede usar el tier gratuito F0.

5.2. Cómo ejecutar el proyecto

Pasos para abrir y ejecutar el proyecto:

1. Abrir el archivo FacturasOCR.sln (o FacturasOCR.csproj) con Visual Studio 2022.
2. Visual Studio debería restaurar automáticamente los paquetes NuGet. Si no, hacer clic derecho en el proyecto → "Restaurar paquetes NuGet".

3. Seleccionar la plataforma de destino en la barra de herramientas. Para Windows: "Windows Machine". Para Android: seleccionar un emulador o dispositivo conectado.
4. Pulsar F5 o el botón de Play para compilar y ejecutar.

También se puede compilar y ejecutar desde la terminal con el comando:

```
dotnet run -f net9.0-windows10.0.19041.0
```

Los paquetes NuGet que usa el proyecto son:

Paquete NuGet	Versión
Azure.AI.FormRecognizer	4.1.0
sqlite-net-pcl	1.9.172
Microsoft.Identity.Client	4.67.2
Microsoft.Maui.Controls	(incluido con .NET 9)

5.3. Configuración de Azure

La app viene con un endpoint y API key por defecto que son los del recurso creado para esta práctica. Si se quiere usar otro recurso de Azure, se pueden cambiar desde la pestaña de Ajustes dentro de la app, o directamente editando el archivo Servicios/ServicioAnálisis.cs.

Para crear un recurso nuevo de Document Intelligence en Azure:

1. Entrar en portal.azure.com con la cuenta de estudiante.
2. Buscar "Document Intelligence" en la barra de búsqueda.
3. Crear un nuevo recurso con el tier F0 (gratuito).
4. Una vez creado, ir a "Keys and Endpoint" y copiar el Endpoint y la Key 1.
5. Pegar esos valores en la pantalla de Ajustes de la app.

6. Funcionalidades principales

6.1. Procesamiento de facturas

La app puede procesar tres tipos de archivos:

- Imágenes (JPG, PNG, BMP, TIFF): se mandan directamente a Azure Document Intelligence.
- PDFs: se mandan a Azure igual que las imágenes, el servicio sabe leerlos.
- XMLs de factura electrónica (FacturaE, XSIG): se parsean localmente sin necesidad de usar Azure, leyendo directamente las etiquetas XML.

Para las imágenes y PDFs, después de recibir el `AnalyzeResult` de Azure (igual que en el ejemplo de clase), la app primero intenta sacar los campos de `Documents[0].Fields` (`VendorName`, `InvoiceId`, etc.), que es lo que en los apuntes se llama "Entities/Fields" (`ShowEntitiesSummary`). Si Azure no los detecta (porque la factura es rara o no está bien escaneada), la app hace una búsqueda manual en los pares clave-valor (`Key-Value Pairs`) y en las tablas como fallback, usando la misma estructura que las funciones `ShowKeyValuePairsSummary` y `ShowTablesSummary` del ejemplo.

Para entender mejor cómo funciona, este es el orden exacto de búsqueda de cada campo (por ejemplo para sacar el total de la factura):

1. Primero se busca en `Documents[0].Fields["InvoiceTotal"]`. Si Azure lo ha detectado como campo del modelo `prebuilt-invoice`, devuelve el valor directamente con su tipo (`Currency`) y su nivel de confianza.
2. Si no lo encuentra ahí (porque la factura está en un formato raro o mal escaneada), se busca en los pares clave-valor (`KeyValuePairs`). Se comparan las claves con palabras como "total", "importe total", "amount due", "grand total", etc.
3. Si tampoco está en los pares clave-valor, se busca en las tablas (`Tables`). Se recorre cada celda buscando la palabra "total" y si la encuentra, coge el valor de la última columna de esa misma fila (que suele ser el importe).
4. Si no se encuentra en ninguna parte, el campo se muestra como "(no detectado)" en la ficha visual.

Este sistema de fallback en cascada es lo que hace que la app funcione razonablemente bien con todo tipo de facturas, no solo con las que Azure reconoce perfectamente.

6.2. Procesamiento por lotes

Además del procesamiento individual, la app tiene un botón "Procesar Lote" que permite seleccionar varios archivos a la vez. Los procesa uno por uno de forma secuencial y al final muestra un resumen con cuántos se procesaron bien y cuántos dieron error. También detecta duplicados automáticamente (compara el nombre de archivo y el contenido) para no guardar la misma factura dos veces.

6.3. Historial y detalle

En la pestaña "Archivo" se muestra la lista de todas las facturas procesadas, ordenadas por fecha. Se puede tocar cualquiera para ver su detalle completo: la imagen original, los campos extraídos (proveedor, número, fecha, total) y el texto completo del OCR. Desde el detalle se puede compartir la información o eliminar la factura de la base de datos.

6.4. Sistema multiidioma

La app soporta 9 idiomas que se pueden cambiar en tiempo real desde la pestaña de Ajustes. Al cambiar el idioma, todos los textos de toda la app se actualizan instantáneamente sin necesidad de reiniciar. Los idiomas disponibles son:

- Castellano (idioma por defecto)
- English
- Català
- Euskara
- Galego
- Aragonés
- Aranés (occitano)
- Asturianu
- Extremeñu

El sistema de traducciones funciona con un servicio estático (ServicioIdiomas) que tiene un diccionario con todas las claves de texto y sus traducciones. Cuando se cambia el idioma se lanza un evento que todas las páginas escuchan para actualizar sus textos.

6.5. Tema claro / oscuro

Desde la pestaña de Ajustes se puede cambiar el tema visual de la aplicación entre tres opciones: Claro, Oscuro o Seguir el sistema (usa el tema que tenga configurado el dispositivo). El tema se guarda en las preferencias de la app y se aplica automáticamente al arrancar, sin necesidad de volver a configurarlo cada vez.

7. Capturas de pantalla

A continuación se muestran capturas de la aplicación funcionando:

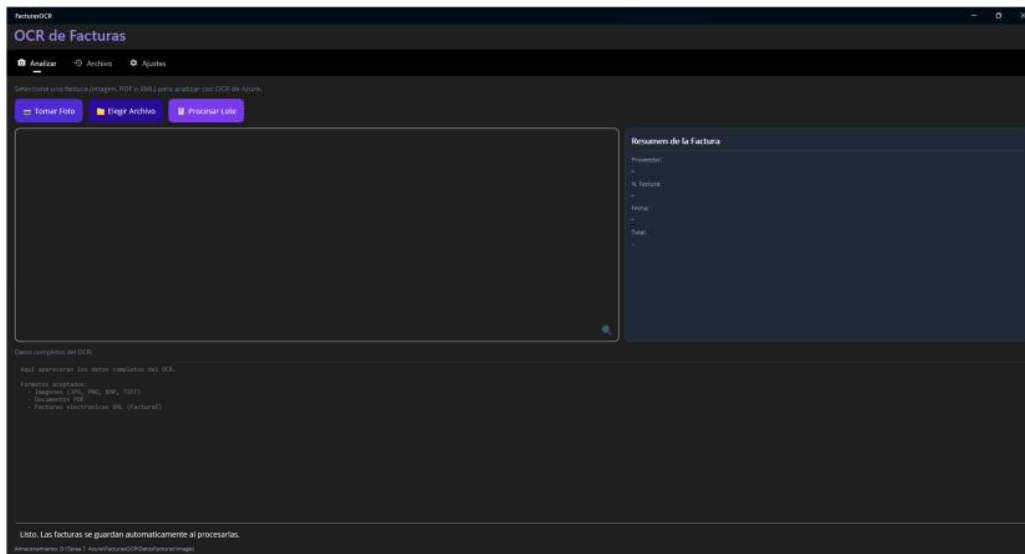


Fig. 2: Pantalla principal antes de procesar ninguna factura

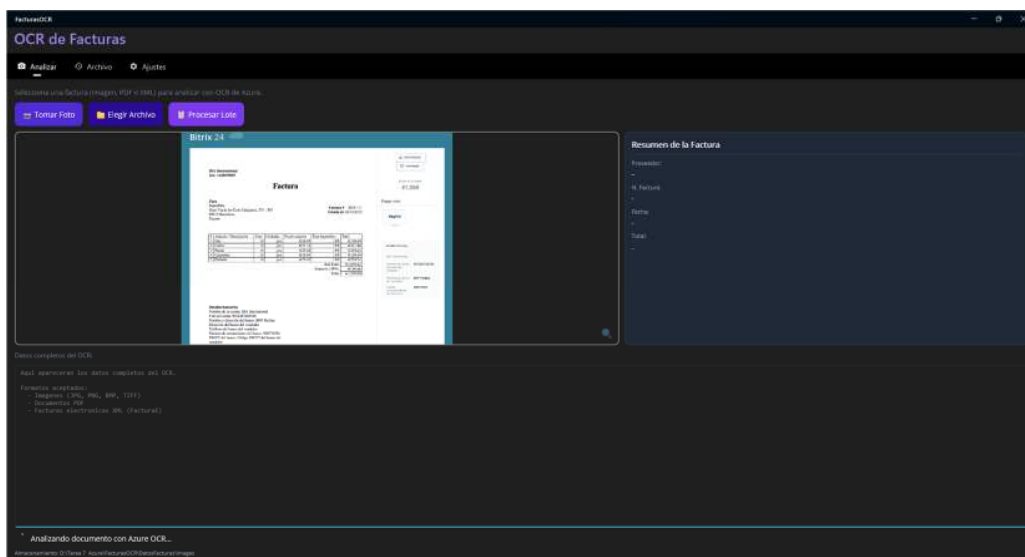


Fig. 3: Procesando una factura con Azure (spinner de carga)

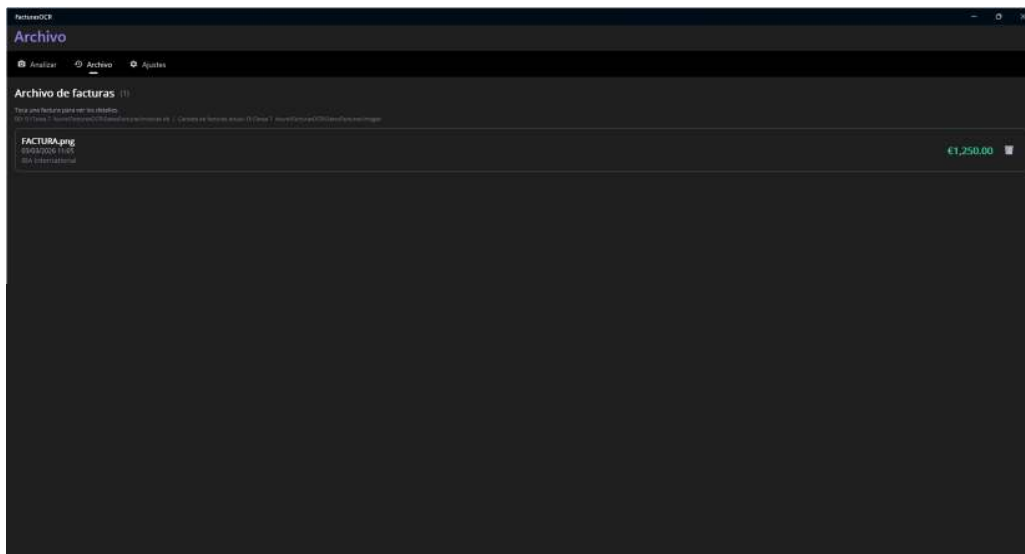


Fig. 6: Historial de facturas guardadas

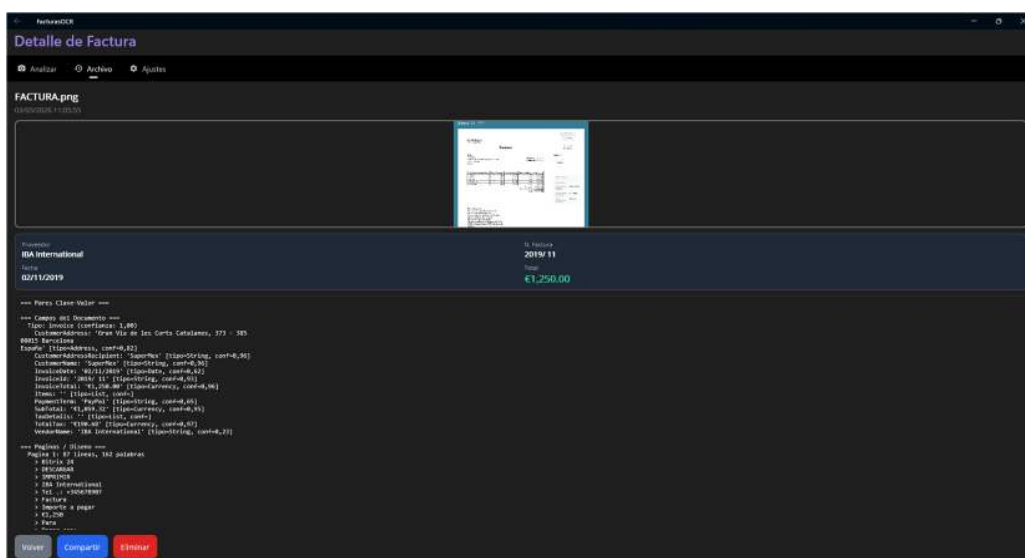


Fig. 7: Detalle de una factura desde el historial

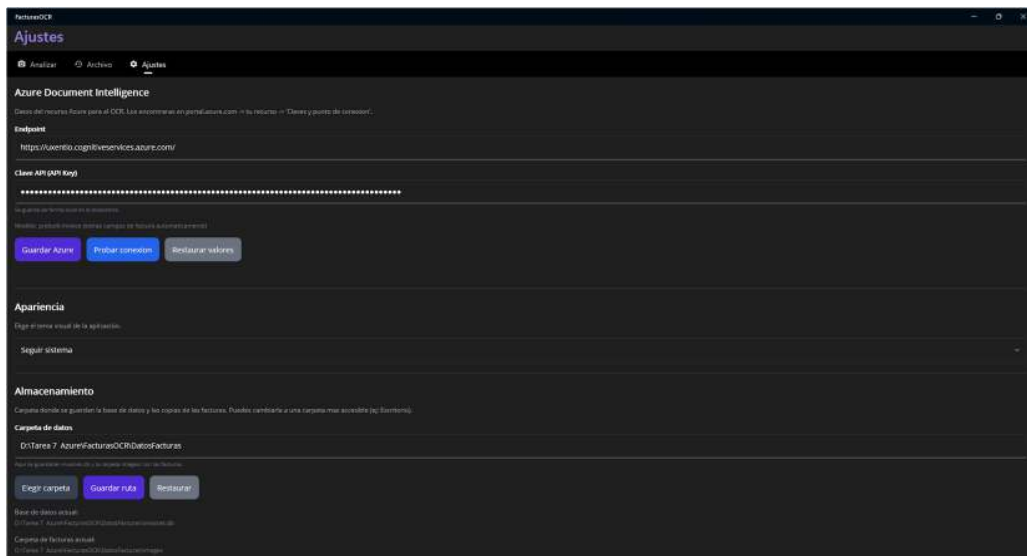


Fig. 8: Pantalla de ajustes: Azure y apariencia

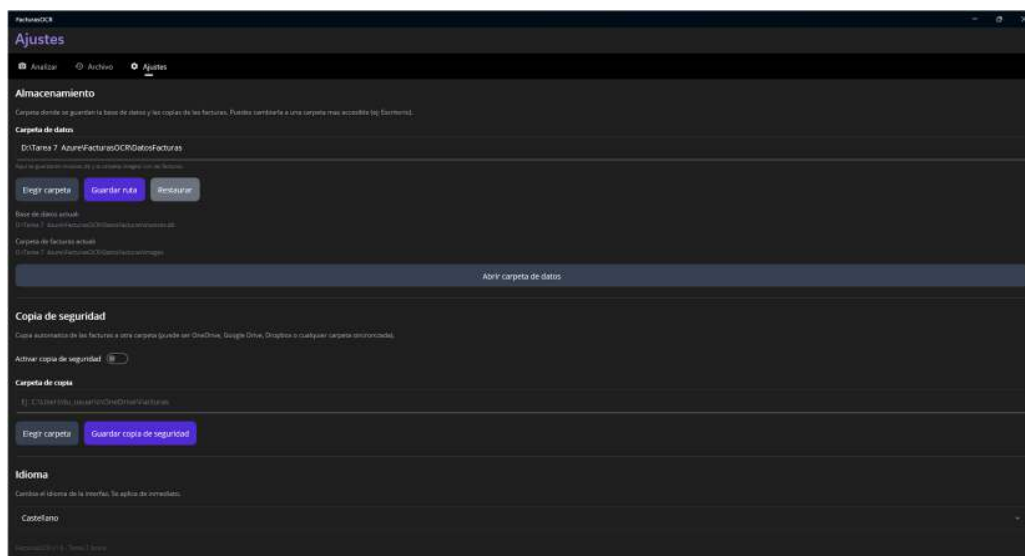


Fig. 9: Pantalla de ajustes: almacenamiento, backup e idioma

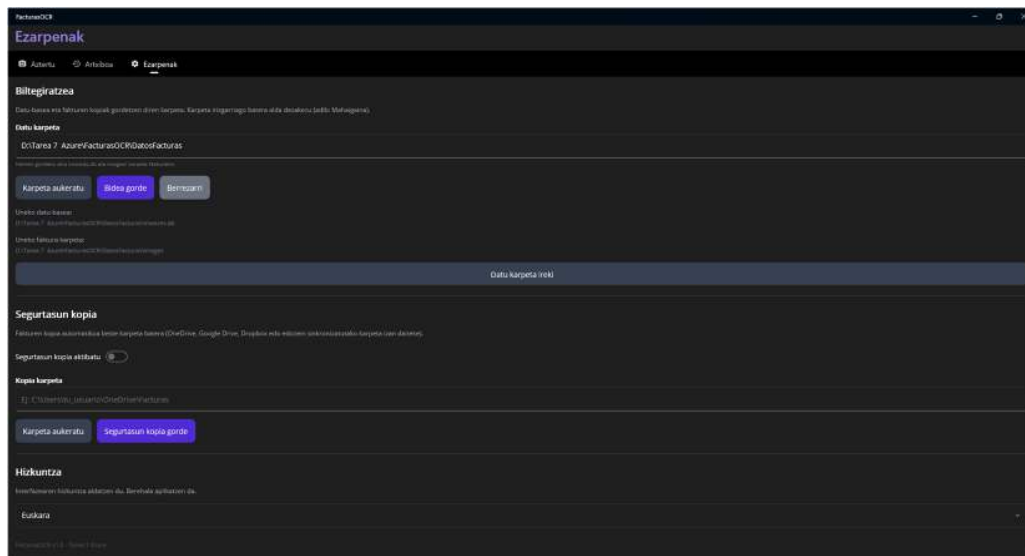


Fig. 10: Ejemplo de la app en euskara

8. Conclusiones

La aplicación cumple con los requisitos del Proyecto 2 de la tarea: permite hacer fotos de facturas con la cámara o elegir archivos, procesarlos con Azure Document Intelligence y almacenar los datos extraídos como texto plano. Los materiales de clase (sobre todo el código de ejemplo del FormRecognizer, el documento de creación del recurso y el glosario de términos) me han sido muy útiles como punto de partida para entender cómo funciona todo el ecosistema de Azure Cognitive Services.

Lo que más me ha costado ha sido la parte de extracción de campos, porque el ejemplo de clase usa "prebuilt-document" (genérico) y yo necesitaba "prebuilt-invoice" (específico de facturas), que devuelve los campos de forma diferente. Además cada factura tiene un formato diferente y a veces Azure no detecta todos los campos, así que implementé un fallback que busca manualmente en los Key-Value Pairs y en las Tables, que es básicamente lo mismo que hacen las funciones Show del ejemplo pero adaptado a mi caso.

También ha sido interesante implementar el soporte multiidioma, que aunque no era un requisito obligatorio, me pareció una buena forma de practicar con eventos y actualización dinámica de la interfaz.

Como posibles mejoras futuras se podría añadir exportación a Excel de las facturas guardadas, o integración con algún servicio de almacenamiento en la nube como OneDrive para sincronizar las facturas entre dispositivos.